

Name (s): Uchenna Osemeka

Course Name: Principles of Software Design

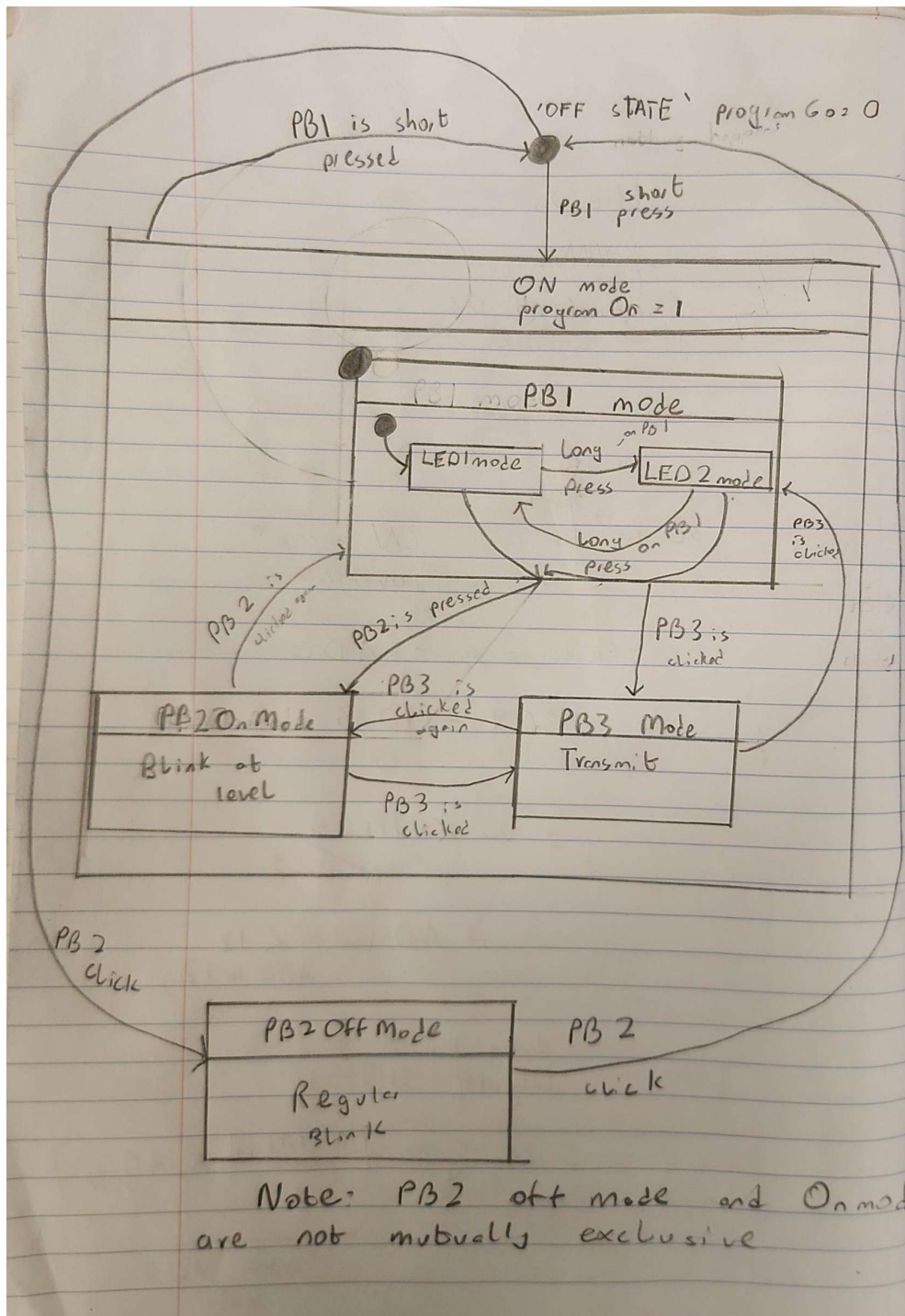
Group: L02-28

Course Code: ENSF 460

Assignment Number: Driver Project 2-Light Control

Submission Date: 5th December, 2025

Design



High Level State Diagram

Our high-level state diagram above shows the general design idea for what flags would trigger our device to change state. Omitted button functionality and specific functions for brevity. Refer to comments in code.

Implementation

Summary of Implementation

We organize files with a control class main which checks using an IOcheck function defined in our ios.h header. ios.c class reads global variables updated by timer2 isr on overflow which debounces earlier cn interrupt. Depending on what global variable(flag) is enabled IOcheck calls the appropriate function defined in itself ios.c and in the ADC.h. ADC.h implements the On Mode. General logic is divided where possible into two primary states programOn=0, and programOn= 1. This division keeps our encapsulation high and proper.

Details of Implementation

Our implementation for a countdown timer truly begins with a main function project3main.

```
int main(void) {  
    /** This is usually where you would add run-once code  
     * e.g., peripheral initialization. For the first labs  
     * you might be fine just having it here. For more complex  
     * projects, you might consider having one or more initialize() functions  
     */  
    builtin_enable_interrupts(); //enable interrupts  
    //AD1PCFG = 0xFFFF; /* keep this line as it sets I/O pins that can also be analog to be digital  
    ADCInit();  
    newClk(500); //set Fosc to 500  
    IOinit(); //initialize I/O  
    InitUART2(); //initialize UART  
  
    while(1) { //keep device on  
        Idle(); //stay in Idle  
        IOcheck(); //perform correct logic  
    }  
    return 0;  
}
```

Here we initialize I/O, and UART transmissions with IOinit(), and InitUART() respectively, then we initialize the ADC module with ADC_init(). Once initialized we have an always on while(1) loop comprising of Idle() and IOcheck(). Idle keeps the system reliably conserving power until an interrupt-a button press or timer overflow- causes the next instruction after Idle, IOcheck to be executed.

Before IOcheck is executed, the CN ISR is called on any button press. The CN ISR checks for a filter initiation with **if(prsLfilter== 1)**, if so it sets the flag for the filter result. After possible filtering, it uses TMR2 to debounce button input on overflow to get a stable read.

```

void __attribute__((interrupt, no_auto_psv)) _CNInterrupt(void) {
    IFS1bits.CNIF = 0; //clear CN interrupt flag
    //possible edge case of multiple cn interrupts erasing prsLfilter
    if(prsLfilter== 1){
        if (TMR3Flag== 1){
            /*If timer has finished and we are filtering, this is a long press*/
            //on PB1 long press, change LED
            //programOn= 1;
            if(LED1Mode==1){
                LED2Mode= 1;
                LED1Mode= 0;
            }
            else if(LED2Mode== 1){
                LED1Mode= 1;
                LED2Mode= 0;
            }
        }

        TMR3Flag= 0;
    }
    else if(TMR3Flag== 0){
        /*If timer has not finished and we are filtering, this is a short press*/
        //go to Off Mode
        programOn= 0;
        LED1Mode= 0;
        LED2Mode= 0;
    }
}

```

On Timer2 overflow, the input is ready and accurate. Depending on what input is read different flags are raised, using if else if logic, as well as contingency to start filter if needed.

```

void __attribute__((interrupt, no_auto_psv)) _T2Interrupt(void) {
    /*This interrupt implements debounce on our push buttons. It ensures we read
    * a stable value, then sets the correct flag for our Iocheck()
    */
    //Don't forget to clear the timer 2 interrupt flag!
    IFS0bits.T2IF = 0;
    T2CONbits.TON = 0; //turn off timer after filter
    if(programOn== 1){
        if(PORTEbits.RB7== 0){
            //on PB1 press while on, filter to decide
            T3CONbits.TCKPS = 2; // set prescaler to 1:64
            TMR3= 0;
            PR3= 11718; //set for 3 seconds give or take
            TMR3Flag= 0;
            T3CONbits.TON = 1;
            prsLfilter= 1; //flag for press Lenght filter
        }
        else if(PORTEbits.RB4== 0){
            //on PB2 press while on, start to blink
            if(PB2OnMode== 1){
                //if already blinking
                PB2OnMode= 0;
            }
        }
    }
}

```

On return to main, IOcheck() is called. This function uses if else if logic to call the appropriate function depending on which flag has been raised.

```
] void IOcheck() {  
[ /*Analyze what flag is set and call corresponding function, an event flag is raised.  
  * IOcheck, the next instruction after Idle() in main, is called. IOcheck uses  
  * if else logic to call the appropriate action defined in timerDelay  
  */  
-   if(programOn== 1){  
[       ONMode();  
        /*  
        if(LED1Mode== 1){  
            ADCtoLED1();  
        }  
        else if(LED2Mode== 1){  
            ADCtoLED2();  
        }  
-   }  
    else if(programOn== 0){  
        ADCEnd();  
        if(PB2OffMode== 1){  
            PB2OffBlink();  
        }  
-   }  
- }
```

PB2Off blink governs pb2 blinking in the off mode.

```
[ void PB2OffBlink() {  
[ //blink in this mode independent of everything else  
    while(PB2OffMode== 1){  
        LATBbits.LATB9^= 1;  
        TMR3= 0;  
        PR3= 12500; //0.5sec timer  
        T3CONbits.TON= 1; //start timer  
        Idle();  
    }  
    if(PB2OffMode== 0){  
        LATBbits.LATB9= 0;  
    }  
}
```

Moving on to where our program spends most of its time ADC.c

```

void ONMode(){
    /*Functionality for On mode. Toggling is done in ISR so this just handles
    starting timer, calculations, and transmission to UART*/
    uint16_t firstTime= 1; // turn TimerOn on first go around
    while(LED1Mode || LED2Mode){
        if(PB2OnMode== 1){
            while(overState==0){ //for blink, force LEDS to state 0
                LATBbits.LATB9= 0;
                LATAbits.LATA6= 0;
                Idle();
            }
        }
        if(guard== 0){ //ensure calculations and displays don't happen too often
            do_ADC();
            //Disp2Hex(ADCvalue);
            uint16_t maxDuty= 309;
            uint16_t maxADC= 1023;
            dutyOn= ((uint32_t)maxDuty * ADCvalue)/maxADC;
            if (dutyOn < 12){
                dutyOn= 12;
            }
            dutyOff= 310- dutyOn;
            if(PB3Mode== 1){
                sendtoPython();
            }
        }
    }
}

```

OnMode handles what happens in the On Mode, here we can transmit for PB3, blink for PB2, and start the PWM. We complete PWM by toggling LED in ISR, more on that later. We use a local variable first time to ensure Timer is only started once. overstate variable is used to force LED state off to simulate blink from PB2. Guard variable is used to make sure we recalculations of ADC and corresponding duty cycle doesn't happen too often preventing visible flicker in our LED1mode

```

/
if(firstTime== 1){
    TICONbits.TON= 1; //start timer
    firstTime= 0;
}
while(1){
    //where the program spends most of its time, periodic breaks-
    //to recalculate and set flags
    Idle();
    if(overState == 0){ //global variable, fires every second and prompts a recalculation
        //guard= 0;
        break;
    }
    /*CONSIDER doing calculation for dutytime in ADC ISR, however
    requirement to transmit makes a 1 second break ideal for now */
}
}

```

Our code spends most of its time switching LED state in our while(1) loop but breaks periodically to recalculate and possibly turn off if blinking

```

void __attribute__((interrupt, no_auto_psv)) _T1Interrupt(void) {
    //This interrupt is used for Pulse Width Modulation
    //
    /*IFS0bits.T1IF = 0;
    T1CONbits.TON= 0; //start timer */
    IFS0bits.T1IF = 0;
    /*USE OF PB2ONmode check makes sure LED isn't forced off when overstate== 0
    in LED1Mode*/
    if(PB2OnMode){//if we are blinking
        if(LED1Mode== 1 && overState== 1){
            if(changeState==0){
                LATBbits.LATB9= 1;
                PR1=  dutyOn; //on time % of 100Hz- 0.01s
                //T1CONbits.TON= 1; //start timer
                changeState= 1;
            }
            else if(changeState== 1){
                LATBbits.LATB9= 0;
                PR1=  dutyOff; //on time % of 100Hz- 0.01s
                //T1CONbits.TON= 1; //start timer
                changeState= 0;
            }
        }
    }
}

```

Actual toggling to do PWM is done in our T1 ISR above. We first check if PB2On mode to prevent blinking when PB2 blink is enabled. Then we use changeState to toggle between the LED states. Duty cycles are calculated as seen earlier.

```

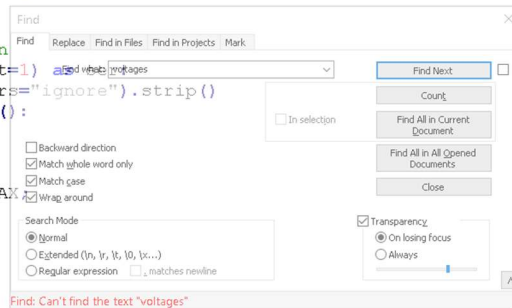
def setup_csv():
    with open(CSV_FILE, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Time (s)", "ADC Value", "Duty Cycle %"])
    print(f"[OK] CSV log created: {CSV_FILE}")

# =====
# SERIAL READER THREAD
# =====
def serial_reader(plot_queue, csv_queue, stop_event):
    while not stop_event.is_set():
        try:
            # Open and close serial each iteration
            with serial.Serial(PORT, BAUD, timeout=1) as ser:
                line = ser.readline().decode(errors="ignore").strip()
                if line and line.lstrip().isdigit():
                    adc = int(line)
                    t = time.time()

                    dutyOn= (maxDuty * adc)/ADC_MAX
                    if (dutyOn < 12):
                        dutyOn= 12

                    D = (dutyOn / maxDuty) * 100
                    #V = (adc / ADC_MAX) * VREF
                    plot_queue.put((t, adc, D))

```



Python code uses threads to log and display ADC values and calculated duty cycle in real time. We use queues for logging and transmissions. And